Nonlinear Dynamics in Computer Graphics: Reaction–Diffusion Systems in Procedural Generation

Ragav Srinivasan

November 13, 2026

Abstract

We start by ti

1 Introduction

1.1 Structure of this report

Section 2 gives the background on reaction–diffusion systems and the phenomena they model. We then discuss a famous result of Alan Turing on reaction–diffusion systems, and finally focus on a simple reaction–diffusion system called the Gray–Scott model. Section 3 is my original contribution: a stability analysis of the Gray–Scott model and a numerical simulation of it. Section 4 gives the conclusion and possible extensions of this project.

2 Background

2.1 Procedural Generation

Procedural generation is a broad set of techniques in computer graphics that is used to create visuals, such as pseudo-realistic textures, terrains and surfaces on the fly. Instead of an artist painting every surface or sculpting every mountain, a developer writes a short program controlled by a few parameters, and the computer produces the detail, often on the fly as the player explores. This makes it possible to build large, *lifelike* landscapes with mountains, rivers and vegetation at very little cost in memory and artist time (Lagae et al. 2010).

Procedural generation is a broad set of techniques in computer graphics that is used to create visuals, such as pseudo-realistic textures, terrains and surfaces on the fly. It eliminates the need for an artists to manually prototype and create their own worlds, Instead, a deterministic function is used to produce the desired visuals, on the fly as the player explores. This makes it possible to build large, *lifelike* landscapes with mountains, rivers and vegetation with little impact to memory. (Lagae et al. 2010).

A well-known example is Minecraft, which uses procedural techniques to generate its worlds and cave systems.

Procedural generation is based on many algorithms. In this section we study one of the most widely used algorithms: *Perlin Noise*.

Origin. Ken Perlin devised his noise function after working on the visual effects for Disney's 1982 film *Tron* at Mathematical Applications Group, Inc. (MAGI), and published it in 1985 (Perlin 1985). His discovery of technique was so pivotal to the industry that it earned him an

Academy Award for Technical Achievement, and still sees widespread in modern day films and video games. (Lagae et al. 2010).

How Perlin noise works. Consider one dimension first. At every integer i we fix a random gradient (slope) g_i. For a point x between i and $i+1$, let $t = x - i \in [0,1]$. Each of the two neighbouring lattice points contributes a straight line through zero with its own slope, and the two lines are blended together:

$$\text{noise}(x) = \big(1 - s(t)\big)\, g_i\, t \;+\; s(t)\, g_{i+1}\, (t-1), \qquad s(t) = 6t^5 - 15t^4 + 10t^3. \tag{2.1}$$

The blending curve s rises smoothly from 0 to 1, and its first and second derivatives vanish at $t = 0$ and $t = 1$, so neighbouring pieces join without visible creases (Perlin 2002). Two properties follow directly from Equation (2.1): the noise is zero at every integer, and its bumps all have roughly the same width, about one lattice cell (**??**a). In two dimensions the idea is the same: each corner $\mathbf{c}$ of a grid square carries a random unit vector $\mathbf{g_c}$, the corner contributes $\mathbf{g_c} \cdot (\mathbf{x} - \mathbf{c})$, and the four contributions are blended with s in each direction (**??**b).

Building textures from noise. A single layer of noise is smooth but bland. Natural surfaces have detail at many scales, so copies of the noise at doubled frequency and halved amplitude are added together:

$$\text{fBm}(\mathbf{x}) = \sum_{j=0}^{n-1} \frac{1}{2^j}\, \text{noise}\big(2^j \mathbf{x}\big)\,, \tag{2.2}$$

which is known as fractal Brownian motion (**??**c). Replacing noise by |noise| in Equation (2.2) gives a "turbulence" function, and feeding it into a simple formula produces a material. Perlin's classic marble, for example, is $\sin\big(x_1 + \text{turbulence}(\mathbf{x})\big)$ (Perlin 1985)

Limitations. Textures created with Perlin noise, and with noise functions in general, are random and have no organised structure (Lagae et al. 2010). Every bump has roughly the same size, but the bumps are placed at random: nothing makes spots evenly spaced or stripes run parallel. This is ideal for clouds, terrain or marble, but if we want a zebra whose stripes organise themselves, a different approach is needed. That approach is the subject of Section 2.2.

2.2 Reaction Diffusion Systems

This brings us to the heart of this report, *Reaction Diffusion Systems*. These are Nonlinear dynamical systems originating from the world of Chemistry that models the intereactions between multiple chemical susbtances and their diffusivity with resepct to space and time.
Diffusion refers to the spread of chemical substances between low and high concentration. It turns out adding an element of diffusion leads to some very interesting pattern formation.
The General Form of Reaction Diffusion Systems is given by:

$$\frac{\partial \mathbf{q}}{\partial t} = D\nabla^2 \mathbf{q} + R(\mathbf{q}), \tag{2.3}$$

- The equation is *dimensionless*
- $\mathbf{q}$ is a vector of unknown vector functions representing the concentrations of an array of chemical substances.
- D gives the diffusivity for each chemical
- $R(\mathbf{q})$ is another vector functions that describes all the chemical reactions.

There are several systems that fit the criteria of Reaction Diffusion Systems. Notably, the activator eqution, some russian equation, etc.

2.3 Turing patterns

[TODO: Define Turing patterns]
Turing patterns are self organized patterns that resembles structures found in nature.
[TODO: Cite turing's 1952 discovery on chemical morphogenesis]

2.4 The Gray-Scott model

The setup for the Gray-Scott model supposes that we have a *Well mixed tank* of chemical U. Then the concentration of U. We are interested in knowing the concentration of the chemical in space and time. For this, we define a vector function $u(x,t)$ that is currently unknown. We assume that chemical U has the same concentration in the spatial domain except for a small patch in the center of the tank where chemical V is introduced into the system.

In chemistry, The relationship between U and V is defined as *Autocatalysis* where the resulting product is a reactant, as is the case with V is in the reaction shown below. The relationship is akin to that of *prey and predator* models.

$$\underbrace{U + 2V \rightarrow 3V}_{V \text{ reacts to } U \text{ to make more of itself}} \qquad \underbrace{V \rightarrow P}_{\text{decay}}$$

The second rule suggests decay for V by turning into an inert product P which means that it becomes a non reactive substance.

$$\frac{\partial U}{\partial t} = -UV^2 + D_u\nabla^2 + f(1-U)$$
$$\frac{\partial V}{\partial t} = UV^2 + D_v\nabla^2 + (k+f)V$$

- D_u and D_v is the diffusivity of chemicals U and V.
- U and V are the unknown vector functions representing the concentrations of the respective chemicals in space and time.
- f is the feed rate of chemical U for V
- k is the decay rate for chemical V.

The broader goal of this project is to explain pattern formation rising from Reaction Diffusion Systems but the narrower goal is to find which parameter values of F and k. Therefore, In the end we will build a parameter map of the Gray Scott Model that can identify which patterns we get.

3 Original Contribution

3.1 Stability analysis

A stability analysis gives us a guess on the behaviour of the system. For instance, a system is said to be stable if solutions move away from the equillibrium point, otherwise an unstable system is when solutions move away from the equillibrium point.

Nonlinear Dynamical Systems are hard to solve analytically.

The point of this stability analysis is to show how adding a diffusion effect can destabilize a system. This is also called as Turing instability.

In this case, we will be doing a stability analysis twice. The first part will prove the Gray Scott Model is stable when we drop diffusion. In the second part we will show that the Gray Scott Model's turing unstable after adding the diffusion constant.

1. Setup the Gray Scott system as follows:

$$\dot{U} = -UV^2 + D_u\nabla^2 + F(1-U)$$
$$\dot{V} = UV^2 + D_v\nabla^2 + (F+k)V$$

2. What are the equillibrium points: From inspection, there are two $(0,0)$ and $(1,0)$.

3. Jacobian of the gray-scott model:

$$J = \begin{pmatrix} \partial_u\dot{U} & \partial_v\dot{U} \\ \partial_u\dot{V} & \partial_v\dot{V} \end{pmatrix} = \begin{pmatrix} -V^2 - F & -2UV \\ V^2 & (F+k)V \end{pmatrix}$$

4. Substitute equillibrium points into jacobian

First equillibrium point:

$$J(0,0) = \begin{pmatrix} -F & 0 \\ 0 & (F+k) \end{pmatrix}$$

$p = Tr(J(0,0)) = k$ and $q = \det(J(0,0)) = -F(F+k)$
what is $p^2 - q$?
[TODO: Write your stability analysis] Therefore, the system is *stable*.

Substituting the second equillibrium point yields:

$$J(1,0) = \begin{pmatrix} -F & 0 \\ 0 & 0 \end{pmatrix}$$

$Tr(J(1,0)) = -F$ and $\det(J(1,0)) = 0$
Therefore, the system is *stable*.

Part 2: We do the same analysis with the diffusion terms added back?

1. Equillibrium points 2. Jacobian 3. Tr J and Det J of the jacobian 4. Is it stable or unstable.

3.2 Simulation of the Gray Scott Model

3.3 Texture Mapping in Blender

[TODO: I will do this section if I have time, for now I will just do the parameter map and stability analysis.]

[TODO: Method details, figures and discussion.]

4 Conclusion

[TODO: Summarise what the stability analysis predicts, how well the simulation matches it, and what this means for someone choosing F and k to generate textures. Possible extensions: varying the diffusion ratio D_u/D_v, comparing the predicted and simulated spot spacing, and running the simulation on a 3D surface as in Turk (1991).]

References

Lagae, Ares et al. (2010). "A Survey of Procedural Noise Functions". In: *Computer Graphics Forum* 29.8, pp. 2579–2600. DOI: 10.1111/j.1467-8659.2010.01827.x.

Perlin, Ken (1985). "An Image Synthesizer". In: *Computer Graphics (Proceedings of SIGGRAPH '85)* 19.3, pp. 287–296. DOI: 10.1145/325165.325247.

— (2002). "Improving Noise". In: *ACM Transactions on Graphics* 21.3, pp. 681–682. DOI: 10.1145/566654.566636.

Turk, Greg (1991). "Generating Textures on Arbitrary Surfaces Using Reaction-Diffusion". In: *Computer Graphics (Proceedings of SIGGRAPH '91)* 25.4, pp. 289–298. DOI: 10.1145/127719.122749.

5 Appendix